پیش‌گزارش آزمایش ششم

شبکه‌های کامپیوتری

آشنایی با سوکت‌نویسی و پروتکل‌های لایه انتقال

نام و نام خانوادگی: آتنا عسکری
شماره دانشجویی: ۴۰۳۳۱۰۳۳
شماره آزمایش: ششم

پرسش

پرسش نخست شیوه ارتباط بین دو واحد در سطح پردازه و دو واحد در سطح شبکه را توضیح داده و تفاوت‌ها و شباهت‌های آنها را بررسی کنید.

پاسخ

ارتباط در سطح پردازه (Process-to-Process) به تبادل داده بین دو برنامه در حال اجرا گفته می‌شود. این ارتباط دو حالت دارد: یا دو پردازه روی یک دستگاه هستند (مثلاً دو نرم‌افزار روی یک کامپیوتر) یا روی دو دستگاه جداگانه (مثلاً مرورگر روی لپ‌تاپ و سرور وب). برای شناسایی یک پردازه مقصد روی شبکه، دو جزء نیاز است: **آدرس IP** (برای یافتن دستگاه) و **شماره پورت** (برای یافتن پردازه خاص روی آن دستگاه، مثل پورت ۸۰ برای وب). مسئولیت این نوع ارتباط بر عهده **لایه انتقال (Transport Layer)** است و پروتکل‌های اصلی آن TCP (ارتباط قابل اعتماد با تصحیح خطا) و UDP (ارتباط سریع بدون تضمین) می‌باشند.

ارتباط در سطح شبکه (Host-to-Host) به جابجایی داده بین دو دستگاه (میزبان) از طریق شبکه فیزیکی گفته می‌شود. این ارتباط توسط **لایه شبکه (Network Layer)** مدیریت می‌شود و پروتکل اصلی آن IP است. در این سطح، هر دستگاه فقط با آدرس IP خود شناخته می‌شود. لایه شبکه داده را به صورت **گام به گام (hop-by-hop)** از طریق روترها به دستگاه مقصد می‌رساند و توجهی ندارد که کدام پردازه روی آن دستگاه باید داده را دریافت کند. پروتکل IP ذاتاً غیرقابل اعتماد است و تضمینی برای رسیدن یا ترتیب صحیح بسته‌ها نمی‌دهد.

تفاوت اصلی این دو سطح در این است که سطح شبکه داده را فقط به دستگاه مقصد می‌رساند، در حالی که سطح پردازه داده را به **پردازه خاص** روی آن دستگاه تحویل می‌دهد. به همین دلیل، سطح شبکه فقط از آدرس IP استفاده می‌کند اما سطح پردازه به آدرس IP به اضافه **شماره پورت** نیاز دارد. همچنین سطح شبکه به صورت **گام به گام** کار می‌کند ولی سطح پردازه به صورت **انتها به انتها** عمل می‌کند.

شباهت اصلی آنها در این است که هر دو در مدل TCP/IP تعریف شده‌اند، هر دو از **سوکت (Socket)** استفاده می‌کنند و در یک ارتباط کامل شبکه، هر دو سطح به صورت پشت سر هم

عمل می‌کنند: ابتدا سطح شبکه بسته را به دستگاه مقصد می‌رساند، سپس سطح پردازه آن را به نرم‌افزار مناسب تحویل می‌دهد.

پرسش

پرسش دوم سوکت چیست؟ انواع آن را توضیح داده و ماژول‌های مرتبط با آن در پایتون را گزارش کنید و بررسی کنید هر یک چه نیازمندی را پوشش می‌دهند.

پاسخ

سوکت چیست؟

سوکت (Socket) یک نقطه پایانی برای ارتباط در شبکه است که به پردازه‌ها اجازه می‌دهد با یکدیگر تبادل داده کنند. مفهوم سوکت از سیستمعامل‌های یونیکس گرفته شده که بر اساس فلسفه "همه چیز یک فایل است" طراحی شده‌اند. در این فلسفه، سوکت مانند یک فایل ویژه عمل می‌کند که می‌توان آن را باز کرد، روی آن نوشت، از آن خواند و در نهایت بست. سوکت‌ها به عنوان رابط بین **لایه کاربردی** و **لایه شبکه** عمل می‌کنند و جزئیات ارتباط فیزیکی را از برنامه‌نویس پنهان می‌سازند.

در اصل، یک سوکت ترکیبی از **آدرس IP** و **شماره پورت** است که یک ارتباط منحصربه‌فرد بین دو پردازه را مشخص می‌کند. این مفهوم برای برقراری ارتباط در سطح پردازه (Process-to-Process) ضروری است، زیرا به لایه انتقال اجازه می‌دهد بداند داده دریافتی باید به کدام برنامه روی دستگاه مقصد تحویل داده شود.

انواع سوکت از نظر خانواده آدرس (Address Family)

خانواده آدرس تعیین می‌کند که سوکت از چه نوع پروتکل‌های شبکه‌ای استفاده کند. مهم‌ترین انواع آن عبارتند از:

- **AF_INET (IPv4)**: این نوع سوکت برای ارتباطات شبکه با استفاده از پروتکل اینترنت نسخه ۴ استفاده می‌شود. برای شناسایی یک سوکت در این خانواده، یک آدرس IPv4 (مثل ۱.۱.۱۶۸.۱۹۲) همراه با یک شماره پورت مشخص می‌شود. این رایج‌ترین نوع سوکت در اینترنت امروزی است.
- **AF_INET6 (IPv6)**: نسخه جدیدتر پروتکل اینترنت که از آدرس‌های ۱۲۸ بیتی استفاده می‌کند. این نوع سوکت برای ارتباطات شبکه با آدرس‌دهی IPv6 طراحی شده است.
- **AF_UNIX**: این نوع سوکت فقط برای ارتباط بین پردازه‌هایی که روی یک دستگاه (یک سیستمعامل) اجرا می‌شوند، کاربرد دارد. برخلاف دو نوع قبلی، از آدرس IP استفاده نمی‌کند بلکه از یک مسیر فایل در سیستم فایل استفاده می‌کند. این روش بسیار سریعتر از سوکت‌های شبکه است زیرا به پشته پروتکل شبکه نیاز ندارد.
- **سایر خانواده‌ها**: خانواده‌هایی مانند AF_PACKET (دسترسی مستقیم به لایه فیزیکی شبکه)، AF_BLUETOOTH (ارتباط از طریق بلوتوث) و AF_VSOCK (ارتباط بین ماشین مجازی و میزبان) نیز وجود دارند که برای کاربردهای خاص طراحی شده‌اند.

انواع سوکت از نظر نوع ارتباط (Socket Type)

نوع سوکت تعیین می‌کند که ارتباط چگونه رفتار کند و چه خدماتی ارائه دهد:

- **SOCK_STREAM (جریان)**: این نوع سوکت یک ارتباط قابل اعتماد، ترتیب‌دار و مبتنی بر اتصال (connection-oriented) فراهم می‌کند. داده‌ها به صورت یک جریان پیوسته از بایت‌ها ارسال می‌شوند و تضمین می‌شود که بسته‌ها به ترتیب و بدون خطا به مقصد می‌رسند. **پروتکل TCP** پیاده‌ساز این نوع سوکت است. این نوع برای برنامه‌هایی که به تحویل مطمئن داده نیاز دارند (مانند مرورگر وب، ایمیل و انتقال فایل) مناسب است.

- **SOCK_DGRAM (دیتاگرام)**: این نوع سوکت یک ارتباط غیرقابل اعتماد، بدون اتصال (connectionless) و مبتنی بر پیام است. داده‌ها در قالب دیتاگرام‌های جداگانه ارسال می‌شوند و تضمینی برای رسیدن، ترتیب یا عدم تکراری بودن وجود ندارد. **پروتکل UDP** پیاده‌ساز این نوع سوکت است. این نوع برای برنامه‌هایی که سرعت برایشان مهمتر از اطمینان است (مانند استریم ویدئو، VoIP و بازی‌های آنلاین) مناسب است.

- **SOCK_RAW (خام)**: این نوع سوکت دسترسی مستقیم به بسته‌های شبکه در سطح IP یا پایین‌تر را فراهم می‌کند. برنامه‌های معمولی نمی‌توانند بسته‌هایی مثل ICMP یا IGMP را با سوکت‌های معمولی پردازش کنند، اما سوکت خام این امکان را می‌دهد. همچنین برنامه می‌تواند سرآیند IP را خودش بسازد. از این سوکت در ابزارهایی مانند ping و traceroute استفاده می‌شود.

- **SOCK_SEQPACKET**: این نوع سوکت مشابه SOCK_STREAM است اما مرزهای پیام را حفظ می‌کند. یعنی داده به صورت بسته‌های جداگانه ارسال می‌شود اما تحویل قابل اعتماد و ترتیب‌دار است. این نوع کمتر از دو نوع اول استفاده می‌شود.

ماژول‌های مرتبط با سوکت در پایتون

پایتون چندین ماژول برای کار با سوکت فراهم می‌کند که هر کدام نیازمندی خاصی را پوشش می‌دهند:

۱. ماژول `socket` (سطح پایه)

این ماژول رابط مستقیم با توابع سوکت سیستمعامل (BSD Socket API) را فراهم می‌کند. تمام انواع سوکت و خانواده آدرس‌ها را پشتیبانی می‌کند و امکان ایجاد سوکت‌های TCP، UDP و RAW را می‌دهد. این ماژول پایه‌ترین و اصلی‌ترین ابزار برای شبکه در پایتون است و تمام ماژول‌های دیگر روی آن ساخته شده‌اند.

- **نیازمندی پوشش داده شده**: فراهم کردن دسترسی مستقیم و کامل به تمام قابلیت‌های سوکت‌های سیستمعامل برای برنامه‌هایی که به کنترل دقیق نیاز دارند.

۲. ماژول `ssl` (امنیت)

این ماژول لایه‌ای از امنیت را روی سوکت‌های معمولی اضافه می‌کند. با استفاده از پروتکل TLS/SSL، داده‌های رد و بدل شده را رمزگذاری می‌کند و هویت سرور را از طریق گواهی‌های دیجیتال تأیید می‌نماید. این ماژول بر پایه OpenSSL پیاده‌سازی شده است.

- **نیازمندی پوشش داده شده**: ایجاد ارتباطات امن و رمزگذاری شده برای برنامه‌هایی مانند وب‌سایت‌های HTTPS، ایمیل‌های امن و تراکنش‌های بانکی.

۳. ماژول `socketserver` (ساده‌سازی سرور)

این ماژول کلاس‌های آماده برای ساخت انواع سرورهای شبکه فراهم می‌کند. توسعه‌دهنده نیازی به مدیریت جزئیات ایجاد سوکت، باینِد کردن و listening ندارد. این ماژول انواع مختلف سرور از جمله:

* `TCPServer` (سرور TCP همزمان)

* `UDPServer` (سرور UDP)

* `UnixStreamServer` و `UnixDatagramServer` (برای سوکت‌های یونیکس)

* نسخه‌های چندنخی (`ThreadingMixIn`) و چندفرآیندی (`ForkingMixIn`) برای مدیریت همزمان چندین کلاینت

– **نیازمندی پوشش داده شده:** کاهش پیچیدگی و زمان توسعه برای برنامه‌هایی که نیاز به یک سرور شبکه با قابلیت مدیریت درخواست‌های همزمان دارند.

۴. ماژول `asyncio` (برنامه‌نویسی ناهمگام)
این ماژول یک چارچوب کامل برای برنامه‌نویسی غیرهمگام (asynchronous) فراهم می‌کند. در حوزه شبکه، توابعی مانند `asyncio.open_connection()` و `asyncio.start_server()` را ارائه می‌دهد که در پس‌زمینه از سوکت‌ها استفاده می‌کنند اما به صورت غیربلوک (non-blocking) کار می‌کنند. این ماژول همچنین از SSL/TLS به صورت یکپارچه پشتیبانی می‌کند.

– **نیازمندی پوشش داده شده:** ساخت برنامه‌های شبکه با کارایی بالا که می‌توانند همزمان هزاران اتصال را بدون نیاز به نخ (thread) یا پردازه (process) جداگانه مدیریت کنند.

۵. کتابخانه‌های شخص ثالث (مانند `anyio`)
کتابخانه‌هایی مانند `anyio` لایه انتزاعی بالاتری روی `asyncio` و سایر Backendهای ناهمگام فراهم می‌کنند. توابعی مانند `anyio.connect_tcp()` و `anyio.create_tcp_listener()` کار با سوکت‌ها را ساده‌تر می‌کنند و قابلیت‌هایی مانند پیاده‌سازی الگوریتم Happy Eyeballs (انتخاب خودکار IPv4 یا IPv6 برای اتصال سریع‌تر) را ارائه می‌دهند.

– **نیازمندی پوشش داده شده:** فراهم کردن یک API یکسان و سطح بالاتر برای برنامه‌نویسی شبکه ناهمگام که روی Backendهای مختلف (`asyncio`، `trio` و غیره) کار کند.

پرسش

پرسش سوم شماره پورت چیست، چه مفهومی را در شبکه پیاده‌سازی می‌کند؟ هر سیستم دارای چند پورت رزرو و عمومی است؟ چگونه پورت‌های فعال یک سیستم را می‌توان دید یا جستجو کرد و یا داده‌ای روی آنها فرستاد. دست‌کم دو ابزار (مانند `nc`، telnet و nmap) و یک سناریو برای هر یک بیاورید. چند نمونه از پورت‌های رزرو را نام ببرید.

پاسخ
شماره پورت چیست و چه مفهومی را پیاده‌سازی می‌کند؟
شماره پورت یک عدد ۱۶ بیتی در بازه ۰ تا ۶۵۵۳۵ است که در **لایه انتقال** برای شناسایی پردازه‌ها (برنامه‌های در حال اجرا) روی یک دستگاه در شبکه استفاده می‌شود. شماره پورت مفهوم **تحویل پردازه به پردازه** را پیاده‌سازی می‌کند؛ به این معنا که آدرس IP داده را به دستگاه مقصد می‌رساند، اما شماره پورت مشخص می‌کند کدام برنامه روی آن دستگاه داده را دریافت کند. همچنین مفهوم **چندگانه‌سازی** را پیاده‌سازی می‌کند؛ یعنی چندین سرویس می‌توانند همزمان روی یک دستگاه با یک آدرس IP کار کنند.

هر سیستم دارای چند پورت رزرو و عمومی است؟

تعداد کل پورت‌ها ۶۵۵۳۶ عدد است. بر اساس تقسیم‌بندی سازمان IANA:

- **پورت‌های رزرو شده (Well-Known Ports)**: شماره‌های ۰ تا ۱۰۲۳ به تعداد **۱۰۲۴ پورت**. این پورت‌ها برای سرویس‌های استاندارد و سیستمی رزرو شده‌اند و معمولاً فقط پردازه‌های دارای دسترسی مدیر می‌توانند روی آنها گوش دهند.
- **پورت‌های عمومی**: کل پورت‌های ۱۰۲۴ تا ۶۵۵۳۵ به تعداد **۶۴۵۱۲ پورت**. این محدوده به دو بخش تقسیم می‌شود: **پورت‌های ثبت شده** (۱۰۲۴ تا ۴۹۱۵۱) برای برنامه‌های کاربردی که ثبت شده‌اند و **پورت‌های پویا** (۴۹۱۵۲ تا ۶۵۵۳۵) که برای استفاده موقتی سمت کلاینت در نظر گرفته شده‌اند.

چگونه پورت‌های فعال یک سیستم را می‌توان دید یا جستجو کرد و داده‌ای روی آنها فرستاد؟
برای مشاهده پورت‌های فعال در سیستم خود، از دستور `netstat` استفاده می‌شود:

- در **لینوکس و مک**: `sudo netstat -tuln`
- در **ویندوز**: `netstat -ano | findstr "LISTENING"`

برای جستجوی پورت‌های یک سیستم راه‌دور و ارسال داده به آنها، از ابزارهای خط فرمان استفاده می‌شود. دو ابزار مهم عبارتند از `nc` (netcat) و `nmap`.

دو ابزار و یک سناریو برای هر یک
ابزار اول: `nc` (netcat) - یک ابزار همه‌کاره شبکه

- **سناریو**: مدیر شبکه می‌خواهد بررسی کند که سرویس SSH روی یک سرور راه‌دور (پورت ۲۲) فعال است.
- **دستور**: `nc -zv 192.168.1.100 22`. اگر پورت باز باشد، پیام موفقیت نمایش داده می‌شود. همچنین با دستور `nc 192.168.1.100 22` می‌توان به سرور متصل شد و نسخه سرویس را از روی بنر (مانند OpenSSH_8.9p1) شناسایی کرد.

ابزار دوم: `nmap` - قوی‌ترین ابزار اسکن پورت

- **سناریو**: مسئول امنیت سازمان می‌خواهد تمام سرویس‌های در حال اجرا روی یک سرور جدید را شناسایی کند تا سطح حمله را ارزیابی نماید.
- **دستور**: `nmap -sV -F 192.168.1.100`. خروجی این دستور پورت‌های باز، نام سرویس و نسخه آن را نشان می‌دهد (مانند پورت ۸۰ با سرویس http و نسخه Apache 2.4.52).

ابزار کمکی: `telnet`

- **سناریو**: توسعه‌دهنده وب می‌خواهد به سرعت بررسی کند سرور وب روی پورت ۸۰ فعال است.
- **دستور**: `telnet example.com 80` سپس تایپ `HEAD / HTTP/1.0` و هدرهای HTTP نمایش داده می‌شود.

چند نمونه از پورت‌های رزرو

سرویس	پورت
سرور به امن اتصال برای SSH	۲۲
ایمیل ارسال برای SMTP	۲۵
دامنه نام سرویس برای DNS	۵۳
رمزگذاری بدون وب برای HTTP	۸۰
رمزگذاری با امن وب برای HTTPS	۴۴۳
داده پایگاه برای MySQL	۳۳۰۶

پرسش

پرسش چهارم برای ساخت سرور در سوکت‌نویسی چه فراخوانی‌هایی نیاز است و هریک چه مقادیری می‌گیرند. یک نمودار توالی (Sequence Diagram) ساخت سوکت و جابجاکردن داده از کلاینت به سرور بکشید.

پاسخ

برای ساخت یک سرور در سوکت‌نویسی با پایتون، **هفت فراخوانی اصلی** مورد نیاز است که به ترتیب اجرا می‌شوند.

اولین فراخوانی `socket.socket()` است که یک سوکت جدید ایجاد می‌کند. این تابع دو ورودی اصلی دارد: اولی **خانواده آدرس** که معمولاً AF_INET برای IPv4 انتخاب می‌شود و دومی **نوع سوکت** که برای ارتباط TCP از SOCK_STREAM استفاده می‌شود. مثال آن server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) است.

دومین فراخوانی `bind()` است که سوکت را به یک آدرس IP و شماره پورت متصل می‌کند. این تابع یک تاپل شامل آدرس IP و پورت را به عنوان ورودی می‌گیرد. برای گوش دادن روی تمام رابط‌های شبکه از آدرس `'0.0.0.0'` استفاده می‌شود. مثال آن server_socket.bind(('0.0.0.0', 8080)) است.

سومین فراخوانی `listen()` است که سوکت را به حالت گوش‌دادن تبدیل می‌کند تا منتظر اتصال کلاینت‌ها باشد. این تابع یک عدد به عنوان ورودی می‌گیرد که نشان‌دهنده **حداکثر تعداد اتصال‌های در انتظار در صف** است، معمولاً عدد ۵ یا ۱۰ استفاده می‌شود. مثال آن server_socket.listen(5) است.

چهارمین فراخوانی `accept()` است که سرور را در حالت منتظر نگه می‌دارد تا یک کلاینت به آن متصل شود. این تابع هیچ ورودی ندارد و **دو خروجی** برمی‌گرداند: یک سوکت جدید برای ارتباط با آن کلاینت خاص و آدرس کلاینت. مثال آن conn, addr = server_socket.accept() است.

پنجمین فراخوانی `recv()` است که داده ارسالی از کلاینت را دریافت می‌کند. این تابع یک عدد به عنوان ورودی می‌گیرد که **حداکثر تعداد بایت قابل دریافت** را مشخص می‌کند، مثلاً ۱۰۲۴ بایت. مثال آن data = conn.recv(1024) است.

ششمین فراخوانی `send()` است که پاسخ یا داده را به کلاینت ارسال می‌کند. این تابع داده را به صورت `bytes` به عنوان ورودی می‌گیرد. مثال آن conn.send(b'Hello Client') است.

هفتمین فراخوانی `close()` است که سوکت را می‌بندد و منابع آن را آزاد می‌کند. **هم برای سوکت ارتباط با کلاینت و هم برای سوکت اصلی سرور** باید این فراخوانی انجام شود. مثال‌های آن conn.close() و server_socket.close() هستند.

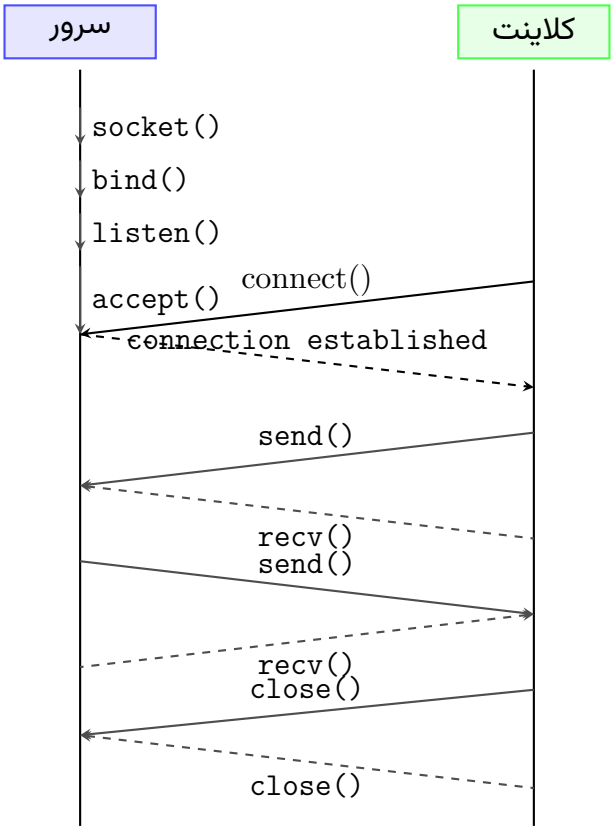
در سمت کلاینت نیز **چهار فراخوانی اصلی** وجود دارد:

- اول `socket()` برای ایجاد سوکت،
- دوم `connect()` برای اتصال به سرور که آدرس IP و پورت سرور را به عنوان ورودی می‌گیرد،
- سوم `send()` و `recv()` برای تبادل داده و
- چهارم `close()` برای بستن اتصال.

نمودار توالی این فراخوانی‌ها به این صورت است که سرور ابتدا `socket`، `bind`، `listen` و سپس `accept` را صدا می‌زند و در `accept` بلوک می‌شود. سپس کلاینت `socket` و `connect` را صدا

می‌زند. پس از برقراری اتصال، `accept` در سرور بازگشت می‌خورد و سوکت جدیدی ایجاد می‌شود. سپس کلاینت با `send` داده می‌فرستد و سرور با `recv` دریافت می‌کند. سرور پس از پردازش با `send` پاسخ می‌دهد و کلاینت با `recv` آن را دریافت می‌کند. در نهایت هر دو با `close` اتصال را می‌بندند.

Sequence Diagram



توضیح روند:

۱. سرور با فراخوانی `socket()`، `bind()` و `listen()` آماده می‌شود.

۲. سرور با `accept()` منتظر اتصال می‌ماند (بلوک می‌شود).

۳. کلاینت با `connect()` درخواست اتصال می‌دهد.

۴. پس از برقراری اتصال، کلاینت با `send()` داده می‌فرستد.

۵. سرور با `recv()` داده را دریافت می‌کند.

۶. سرور با `send()` پاسخ می‌دهد.

۷. کلاینت با `recv()` پاسخ را دریافت می‌کند.

۸. هر دو با `close()` اتصال را می‌بندند.

کد کامل سرور:

```python
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('0.0.0.0', 8080))
server.listen(5)

conn, addr = server.accept()
data = conn.recv(1024)
conn.send(b'Message received')

conn.close()
server.close()
```

کد کامل کلاینت:

```python
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('127.0.0.1', 8080))
client.send(b'Hello')

response = client.recv(1024)
client.close()
```

منابع

- Kurose, J. F. & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson.
- Stevens, W. R. (1994). *TCP/IP Illustrated, Volume 1*. Addison-Wesley.
- Python Software Foundation. (2025). *socket — Low-level networking interface*. Python 3.12 Documentation.
- IANA. *Service Name and Transport Protocol Port Number Registry*.
- Cisco Networking Academy. (2020). *CCNA Introduction to Networks*. Cisco Press.